

Single-image georeferencing of a MicaSense frame over flat ground

Mapping image pixels to ground coordinates (E, N) from one capture — no second image, no panel, no GCP

Developed by Ali Lotfi, in collaboration with Claude Opus 4.8

1. The idea

An orthomosaic carries a 6-parameter *affine* geotransform because orthorectification has already resampled the imagery onto a planar map grid, so pixel $\rightarrow$ ground is linear. A *raw* frame is a central (perspective) projection of the 3-D scene: a pixel defines a *ray*, not a point, and a ground coordinate is obtained only by intersecting that ray with a surface. For an agricultural field the surface is well approximated by a plane, and then the pixel $\leftrightarrow$ ground map of one frame is an *exact* 3×3 projective homography (8 parameters) — the projective generalisation of the ortho's affine. A tilt away from nadir contributes the off-diagonal (keystone) terms that an affine cannot represent.

2. Camera model from the image metadata

Every parameter below is read directly from the file's XMP/EXIF.

Intrinsics (pixels). With $\rho = \text{FocalPlaneXResolution}$ (px mm $^{-1}$), focal length $f = f_{\text{mm}}\rho$ and principal point $(c_x, c_y) = (\text{pp}_x\rho, \text{pp}_y\rho)$, giving

$$\mathbf{K} = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

We use the OpenCV camera frame: x right, y down, z along the optical axis (downward at nadir).

Lens distortion. Brown model, radial k_1, k_2, k_3 and tangential p_1, p_2 (`Camera:PerspectiveDistortion`, in that order).

Exterior orientation. The projection centre is $\mathbf{C} = (E_0, N_0, h)^\top$, where (E_0, N_0) is the GPS longitude/latitude projected into the local UTM zone and h the GPS altitude. Note: MicaSense records `GPSPAltitude` as height above the WGS84 *ellipsoid*; this is automatically converted to orthometric (MSL) height via the EGM96 geoid grid so that it is comparable with DEM ground elevations. The orientation is the rotation $\mathbf{R}$ that takes camera axes into the world East-North-Up (ENU) frame, built from the EXIF Yaw/Pitch/Roll (ψ, θ, φ) :

$$\mathbf{R} = \mathbf{C}_{en} \mathbf{C}_{nb}(\psi, \theta, \varphi) \mathbf{R}_{cb}, \quad \mathbf{R}_{cb} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{C}_{en} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

Here $\mathbf{R}_{cb}$ is the fixed look-down camera $\rightarrow$ body map, $\mathbf{C}_{en}$ swaps the navigation NED frame to ENU, and $\mathbf{C}_{nb} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\varphi)$ is the body $\rightarrow$ NED rotation (Bäumker convention, ψ yaw, θ pitch, φ roll).

3. From a pixel to ground coordinates (E, N)

For a pixel (u, v) :

(i) **Normalise:** $x_d = (u - c_x)/f$, $y_d = (v - c_y)/f$.

(ii) **Undistort** — recover the ideal (x, y) by inverting the Brown model, with $r^2 = x^2 + y^2$,

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2),$$

$$y_d = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy,$$

solved by a few fixed-point iterations (MicaSense distortion is sub-pixel except at the edges).

(iii) **Form the world ray:** the camera-frame direction $\mathbf{d}_c = (x, y, 1)^\top$ rotates to $\mathbf{d} = \mathbf{R} \mathbf{d}_c$.

(iv) **Intersect the ground plane** $Z = Z_g$ (with Z_g the field elevation). The ray is $\mathbf{P}(t) = \mathbf{C} + t \mathbf{d}$; the plane fixes the scale,

$$\boxed{t = \frac{Z_g - C_z}{d_z}, \quad \begin{bmatrix} E \\ N \end{bmatrix} = \begin{bmatrix} C_x \\ C_y \end{bmatrix} + t \begin{bmatrix} d_x \\ d_y \end{bmatrix}.$$

That pair (E, N) is the ground coordinate of the pixel, in UTM.

Equivalent closed form (the matrix). For a plane the entire map is a single homography. Place the local ground origin at the nadir point (plane $Z = 0$, camera at $(0, 0, H_A)$ with $H_A = h - Z_g$). Then

$$s(u, v, 1)^\top = \mathbf{K} [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}] (E, N, 1)^\top, \quad \mathbf{t} = -H_A \mathbf{r}_3,$$

where $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ are the columns of $\mathbf{R}^\top$ (world→camera). Inverting the 3×3 matrix gives (E, N) directly from (u, v) ; absolute coordinates are this plus (E_0, N_0) . The relation is exact for a plane and applies to *undistorted* pixels (do step ii first).

4. What to expect

Because the model is geometrically exact for a planar surface, pixel ↔ ground round-trips to numerical zero. Three practical limits remain. **(a) Vertical datum:** the GPSAltitude in the metadata is an ellipsoid height (WGS84); DEM ground elevations are orthometric (above the geoid/MSL). The script converts automatically using EGM96 so the two are in the same datum. If the geoid grid is unavailable, the fallback is to treat GPS as orthometric with a warning. For absolute-certainty workflows, give the AGL directly via `--agl` to sidestep the datum question entirely. **(b) Absolute position** inherits the onboard GNSS uncertainty — roughly 1–2 m for a standard fix (the EXIF `GPSXYAccuracy/GPSZAccuracy` report it), removable only with RTK/PPK or a ground control point. **(c) Canopy parallax:** at low flight height a differential crop canopy behaves like relief and shifts edge pixels by about $(h_{\text{canopy}}/H_A) \times (\text{edge distance})$ — decimetre-scale for a 1 m canopy at 22 m AGL, vanishing toward nadir. A *uniform* field slope is harmless, since a homography handles any tilted plane.

5. Running it

Dependencies: `numpy`, `pyproj`, `Pillow`, `tiff file` (and `rasterio` only for GeoTIFF output).

Print the homography and self-checks:

```
python micasense_georef.py IMG_0010_4.tif --ground-elev 481.5
```

Write a north-up GeoTIFF (UTM zone auto-selected from the GPS):

```
python micasense_georef.py IMG_0010_4.tif --ground-elev 481.5 --gsd 0.012 --out out.tif
```

Alternatively, give the camera height above ground directly (sidesteps vertical-datum ambiguity):

```
python micasense_georef.py IMG_0010_4.tif --agl 22.0 --out out.tif
```

`--ground-elev` is the field elevation in metres (orthometric/MSL from a DEM; it sets the footprint's scale and position), and `--gsd` the output pixel size in metres. `--agl` is an alternative that gives the absolute height above ground in metres (from the flight plan or other source); exactly one of `--ground-elev` or `--agl` is required. Run the script per band *after* band-to-band alignment to build a georeferenced multispectral stack.